الدليل البرمجي التفصيلي: كيف تم فحص وتدقيق بيانات MRI كوداً بسطر

تفكيك وشرح الكود المصدري لسكريبت `validate_all_cases.py` و `create_splits_and_inventory.py`

الملفات المشروحة: validate_all_cases.py | create_splits_and_inventory.py **المكتبات:** NiBabel, NumPy, Pathlib, Json

الجمهور المستهدف: مهندسو الذكاء الاصطناعي (AI / ML Engineers)

1. التحدي الهندسي الأساسي: لماذا لم نستخدم `nib.load().get_fdata()` مباشرة؟

في مشاريع الرؤية الحاسوبية العادية (2D Computer Vision)، فتح 484 صورة أمر بديهي. لكن في بيانات الرنين المغناطيسي ثلاثية الأبعاد (3D/4D MRI):

- كل مريض يملك مصفوفة بحجم `[4 ,155 ,240 ,240]` بدقة Float32، أي ما يقارب **71 ميجابايت في الذاكرة لكل حالة غير مضغوطة**.
- لو قمنا بتحميل 484 حالة دفعة واحدة في الـ RAM: $$484 \times 71 \text{ MB} \approx 34.4 \text{ Gigabytes of RAM!}$$ وهذا سيؤدي حتماً إلى انهيار البرنامج بسبب نفاد الذاكرة (Out-Of-Memory Crash).

الحل الهندسي: تقنية البث الكسول (Lazy Array Proxy via `img.dataobj`)
مكتبة `NiBabel` توفر كائناً عبقرياً اسمه `dataobj`. هذا الكائن يقرأ الـ Header وأبعاد الصورة فوراً **دون تحميل بكسل واحد في الرام**، ويسمح لنا بتقطيع البيانات وقراءة قناة واحدة فقط عند الحاجة ثم تفريغها فوراً من الذاكرة (Garbage Collection).

2. تفكيك الكود: كيف تم فحص الحالات سطراً بسطر؟

كل الفحوصات تمت داخل دالة مركزية اسمها `inspect_case(root, name, image_rel, label_rel)` في ملف `validate_all_cases.py`. دعنا نشرح كل فحص بالكود الفعلي:

الفحص 1: التأكد من وجود الملفات وأمان المسارات (Path Security)

```
# 1. تحويل المسار النسبي إلى مسار مطلق والتحقق من عدم اختراق المجلد
img_path = (root / image_rel).resolve()
try:
    img_path.relative_to(root.resolve())
except ValueError:
    result["errors"].append("SECURITY: image path escapes dataset root")
    return result

if not img_path.is_file():
    result["errors"].append(f"MISSING IMAGE: {{image_rel}}")
    return result
```

ماذا يفعل الكود؟ يدمج مسار المجلد الجذري مع المسار النسبي المذكور في `dataset.json`، ثم يتأكد عبر `is_relative_to` أن المسار لا يحاول الصعود خارج المجلد، ثم يتحقق عبر `is_file()` من وجود الملف فعلياً على القرص الصلب.

الفحص 2: قراءة الهيكل واكتشاف الملفات التالفة (Corrupt File Detection)

```
# 2. قراءة ترويسة الملف واكتشاف الملفات المعطوبة NIfTI
try:
    img = nib.load(str(img_path))
    sh = img.shape  # Tuple يقرأ الأبعاد فوراً من الـ Header دون لمس الفوكسلات
except Exception as exc:
    result["errors"].append(f"CORRUPT IMAGE ({{type(exc).__name__}}): {{exc}}")
    return result
```

ماذا يفعل الكود؟ يحيط عملية الفتح بكتلة `try-except`. إذا كان الملف مضغوطاً بشكل غير كامل (Incomplete Gzip) أو تالف البايتات (Corrupted Header)، يمسك الاستثناء ويسجل الحالة كـ FAIL مع توثيق اسم الخطأ الدقيق بدلاً من أن يتوقف البرنامج فجأة.

الفحص 3: التحقق من الأبعاد والقنوات الأربع [X, Y, Z, 4]

```
# 3. التأكد أن الصورة 4D وأن القناة الأخيرة تحتوي بالضبط 4 أطياف رنين
result["image_shape"] = list(sh)
if len(sh) != 4:
    result["errors"].append(f"SHAPE: expected 4-D, got {{len(sh)}}-D {{sh}}")
elif sh[3] != 4:
    result["errors"].append(f"CHANNELS: expected 4, got {{sh[3]}}")
```

ماذا يفعل الكود؟ يتحقق أن `len(sh) == 4` (حجم ثلاثي الأبعاد + بعد القنوات)، وأن `sh[3] == 4` للتأكد من وجود القنوات الأربع كاملة (FLAIR, T1, T1ce, T2) دون نقصان أي تسلسل تصويري.

الفحص 4: فحص التباعد الوكسلي الفيزيائي (Voxel Spacing / Zooms)

```
# 4. التأكد من أن المسافات الفيزيائية بين النقاط صالحة وموجبة
zooms = [float(z) for z in img.header.get_zooms()[:3]]
result["spacing_mm"] = zooms
if any(not np.isfinite(z) or z <= 0 for z in zooms):
    result["errors"].append(f"SPACING: invalid voxel size {{zooms}}")
```

ماذا يفعل الكود؟ يستخرج مقاس الفوكسل بالميليمتر من الترويسة `img.header.get_zooms()`. يتأكد أن القيم أرقام حقيقية موجبة تماماً (ليست صفرية ولا سالبة ولا لانهائية)، لأن أي قيمة شاذة هنا ستفسد حساب أحجام الأورام بالـ mL لاحقاً.

الفحص 5: كشف أرقام NaN و Infinity بدون استنزاف الذاكرة (Channel Streaming)

```
# 5. بث القنوات قناة تلو الأخرى وفحص القيم غير المنتهية
if len(sh) == 4:
    for c in range(sh[3]):  # حلقة تكرار على القنوات 0, 1, 2, 3
        channel = np.asarray(img.dataobj[..., c])  # تحميل قناة واحدة فقط في الرام!
        if not np.isfinite(channel).all():
            n_bad = int(np.sum(~np.isfinite(channel)))
            result["errors"].append(
                f"NON-FINITE INTENSITY: channel {{c}} has {{n_bad}} bad voxels"
            )
```

ماذا يفعل الكود؟ هنا تكمن قوة الحل الهندسي: بدلاً من قراءة الحجم الرباعي كاملاً، نستخدم `img.dataobj[..., c]` لقراءة قناة واحدة فقط (حجمها 17 ميجابايت بدلاً من 71 ميجابايت)، ثم نفحصها بـ `np.isfinite()`، وبمجرد انتهاء القناة تحرر الذاكرة فوراً للانتقال للقناة التالية.

الفحص 6: فحص تطابق مصفوفة الـ Affine بين الصورة والقناع (Spatial Alignment)

```
# 6. التحقق من أن الصورة والقناع متطابقان في نفس زاوية وإحداثيات الفضاء
if not np.allclose(img.affine, msk.affine, atol=1e-3, rtol=1e-5):
    diff = float(np.max(np.abs(img.affine - msk.affine)))
    result["errors"].append(f"AFFINE MISMATCH: max delta = {{diff:.6f}}")
```

ماذا يفعل الكود؟ مصفوفة `affine` بحجم 4×4 تحدد دوران وميلان وموضع الرأس في جهاز الرنين. الكود يختبر رياضياً بـ `np.allclose` أن مصفوفة الصورة والقناع متطابقتان بنسبة سماحية صارمة (${10^{-3}}$). لو كان هناك أي انحراف، لكان القناع سيرسم الورم في مكان مختلف عن مكانه الحقيقي!

الفحص 7: حصر الفئات بدقة والتأكد من خلوها من الأرقام الكسرية

```
# 7. قراءة مصفوفة القناع والتحقق من الفئات الفريدة
m = np.asarray(msk.dataobj)
unique_vals = np.unique(m)
int_vals = np.round(unique_vals).astype(np.int32)
result["labels_present"] = [int(v) for v in int_vals]

# أ) التأكد أن كل القيم أرقام صحيحة وغير كسرية
if not np.all(unique_vals == np.floor(unique_vals)):
    result["errors"].append("MASK VALUES: non-integer label values")

# ب) التأكد أن الفئات تنتمي حصراً إلى {0, 1, 2, 3}
bad_labels = set(int_vals.tolist()) - {0, 1, 2, 3}
if bad_labels:
    result["errors"].append(f"LABELS OUT OF RANGE: unexpected labels {{sorted(bad_labels)}}")
```

ماذا يفعل الكود؟ يستخرج كل الأرقام الموجودة في القناع عبر `np.unique()`. ثم يتأكد بشرطين: 1. أن القيم متطابقة مع تقريبها للأسفل `unique_vals == np.floor(unique_vals)`، لمنع الأقنعة الناتجة عن استيفاء خطي خاطئ ينتج أرقاماً كسرية مثل 1.4 أو 2.8. 2. بطرح المجموعات الرياضية `set(int_vals) - {0, 1, 2, 3}`، يتأكد ألا يوجد أي رقم غريب مثل 4 أو 255.

3. كود منع تسرب البيانات وتقسيم المرضى (create_splits_and_inventory.py)

التقسيم تم بالكامل على مستوى **معرف المريض (Case ID)** وليس الشرائح. إليك الكود الفعلي المطبق:

```
# 1. تثبيت الـ Random Seed الرياضي لإعادة الإنتاج التامة
SEED = 42
rng = np.random.default_rng(SEED)

# 2. ترتيب المعرفات أولاً ثم خلطها عشوائياً
shuffled_ids = np.array(sorted(case_ids))
rng.shuffle(shuffled_ids)

# 3. حساب نسب التقسيم (70% Train, 15% Val, 15% Test)
n_train = int(round(num_total * 0.70))  # 339 مريض
n_val = int(round(num_total * 0.15))    # 73 مريض
n_test = num_total - n_train - n_val    # 72 مريض

train_ids = sorted(shuffled_ids[:n_train])
val_ids = sorted(shuffled_ids[n_train:n_train + n_val])
test_ids = sorted(shuffled_ids[n_train + n_val:])

# 4. صمامات الأمان الصارمة: التحقق الرياضي من انعدام أي تقاطع (Disjoint Sets)
assert len(set(train_ids) & set(val_ids)) == 0, "Leakage between Train and Val"
assert len(set(train_ids) & set(test_ids)) == 0, "Leakage between Train and Test"
assert len(set(val_ids) & set(test_ids)) == 0, "Leakage between Val and Test"
assert len(train_ids) + len(val_ids) + len(test_ids) == num_total
```

لماذا يعتبر هذا الكود حاسماً للمسابقة؟

استخدام `set(train_ids) & set(val_ids) == 0` يضمن رياضياً عدم وجود أي مريض مشترك بين مجموعات التدريب والتحقق والاختبار. إذا كان هناك تسريب لبيانات المريض الواحد، سيعطي النموذج أرقام دقة مضللة تفشل تماماً عند التقييم على الحالات المحجوبة (Unseen Test Set).

4. كيف تم توليد تقرير النتائج الموثق (validation_report.json)؟

```
# كتابة الملخص والإحصائيات بصيغة JSON آلياً قابلة للقراءة
summary = {{
    "expected_training": manifest.get("numTraining"), # 484
    "actual_training_entries": n_total,
    "passed": n_passed,                              # 484
    "failed": n_failed,                              # 0
    "training_duplicates": duplicates,               # []
    "train_test_id_overlap": overlaps,               # []
}}
report = {{"dataset": "BRATS", "summary": summary, "cases": results}}

args.out.write_text(json.dumps(report, indent=2, ensure_ascii=False), encoding="utf-8")
```

الخلاصة الهندسية لزميلك:

هذا الكود بنى خط دفاع أولي صلب جعلنا واثقين بنسبة 100% أن قاعدة البيانات خالية من أي تشوهات رقمية أو هندسية، وأن مسار التدريب لا يمكن أن يتعطل بسبب خطأ قراءة في ملف واحد في منتصف التدريب!